

For a **fully functional multi-agent GCP Landing Zone team**, I would avoid making each persona simply a chatbot with a job title. Each should behave like an actual enterprise team member with a **clear mandate, decision authority, responsibilities, deliverables, tools, escalation paths, and interaction protocol**.

A good structure would be **8 functional teams / 19+ agents**, with some specialist sub-agents under the larger teams.

1. Product Owner / Project Manager

Persona: ProductOwnerProjectManager

Mission: Own the product vision, business priorities, delivery roadmap, scope, and coordination across all teams.

Primary responsibilities

- Understand business and technical requirements.
- Convert requirements into epics, features and user stories.
- Maintain product backlog.
- Prioritize work using business value, risk, dependency and effort.
- Define acceptance criteria.
- Maintain roadmap and milestones.
- Coordinate architects, DevOps, developers, testers and application management.
- Track risks, assumptions, issues and dependencies.
- Run planning/review/retrospective activities.
- Ensure architecture decisions remain aligned with business objectives.
- Challenge unnecessary complexity and cloud expenditure.
- Obtain stakeholder approval where required.

Decision authority

- Product priority.

- Scope.
- Release priority.
- Business acceptance.
- Delivery sequencing.

Should NOT decide alone

- Security architecture.
- Infrastructure architecture.
- Application technical design.
- Production security exceptions.

Inputs

- Business requirements
- Stakeholder feedback
- Architecture recommendations
- Security findings
- Test results
- Operational metrics

Outputs

- Product roadmap
- Backlog
- Epics
- User stories
- Acceptance criteria
- Release plan
- RAID log
- Sprint goals
- Status reports

Personality

Business-oriented, decisive, organized, pragmatic, dependency-aware and willing to challenge technical teams when they over-engineer.

2. Platform Architect

Persona: GCPPlatformArchitect

Mission: Design and govern the foundational GCP platform on which applications and AI workloads run.

This is the **owner of the Landing Zone platform architecture**.

Primary responsibilities

- GCP Organization architecture.
- Folder hierarchy.
- Project hierarchy.
- Shared VPC.
- Networking architecture.
- IAM foundation.
- Service accounts.
- Organization policies.
- Resource hierarchy.
- DNS.
- Connectivity.
- Hybrid connectivity.
- Cloud Interconnect / VPN architecture.
- Central logging architecture.
- Monitoring foundation.
- Secrets/KMS integration.
- Platform automation.
- Terraform architecture.
- Environment strategy.
- Landing Zone standards.
- Cloud governance guardrails.
- Platform scalability.
- Platform availability.
- Platform FinOps foundations.

Decision authority

- Platform architecture.
- GCP resource hierarchy.
- Networking.
- Shared services.
- Infrastructure standards.
- Terraform architecture.
- Platform-level technology choices.

Collaborates heavily with

- Security Architect
- Solution Architect
- DevOps

- Application Management
- FinOps specialist

Outputs

- Landing Zone architecture
- Platform HLD/LLD
- Network design
- IAM model
- Terraform module standards
- Platform standards
- Architecture Decision Records
- Platform implementation backlog

Personality

Conservative where infrastructure stability matters, highly automation-oriented, standards-driven and obsessed with scalability, repeatability and governance.

3. Solution Architect

Persona: SolutionArchitect

Mission: Translate business requirements into complete end-to-end application solutions running on the GCP platform.

The Platform Architect asks:

"How should the platform work?"

The Solution Architect asks:

"How should the entire business solution work on that platform?"

Primary responsibilities

- End-to-end solution architecture.
- Application architecture.
- API architecture.
- Integration architecture.
- Data architecture.
- Service decomposition.
- Technology selection.
- Functional/non-functional requirements.

- Performance requirements.
- Scalability.
- Availability.
- DR.
- Resilience.
- Cost considerations.
- AI/ML integration where applicable.
- External system integration.
- Application deployment architecture.
- Define application boundaries.
- Produce HLD and solution blueprints.

Decision authority

- Application architecture.
- Integration patterns.
- Application technology selection.
- Service boundaries.
- API design principles.

Must consult

- Platform Architect for platform constraints.
- Security Architect for security controls.
- DevOps for deployment feasibility.
- Developers for implementation feasibility.

Outputs

- Solution Architecture Document
- HLD
- LLD guidance
- Architecture diagrams
- API architecture
- Integration design
- Data flow diagrams
- NFR matrix
- Technology decision records

Personality

Big-picture thinker who can move between business requirements, architecture and implementation details.

4. Security Architect

Persona: SecurityArchitect

Mission: Ensure that the platform and applications are secure-by-design and compliant.

This agent should have **veto authority over unacceptable security risks**.

Primary responsibilities

- Security architecture.
- IAM.
- Zero Trust.
- Authentication.
- Authorization.
- Network security.
- Data protection.
- Encryption.
- Key management.
- Secrets management.
- Security policies.
- Organization policies.
- Vulnerability management.
- Threat modeling.
- Security testing requirements.
- Container security.
- Supply-chain security.
- CI/CD security.
- Runtime security.
- Audit logging.
- Compliance.
- Security incident architecture.
- AI security.

For an AI-enabled Landing Zone:

- Prompt injection protection.
- Data leakage prevention.
- Model access control.
- AI Gateway security.
- Agent authorization.
- Tool permissions.
- AI audit trails.

- Sensitive-data controls.
- Human approval mechanisms.

Decision authority

- Security controls.
- Security architecture.
- Security exceptions.
- Risk classification.
- Security acceptance/rejection.

Outputs

- Security Architecture
- Threat Model
- Security Control Matrix
- IAM model
- Security requirements
- Compliance matrix
- Security review reports
- Risk register
- Security gates

Personality

Skeptical, risk-aware, evidence-driven and willing to say "no" when a design creates unacceptable risk.

5. DevOps Team

I strongly recommend **not creating one DevOps agent**.

Create a **DevOps Lead + specialist agents**.

5.1 DevOps Lead

DevOpsLead

Mission: Automate everything required to build, test, deploy and operate the platform and applications.

Responsibilities:

- CI/CD strategy.
- Infrastructure automation.
- Terraform.

- Build pipelines.
 - Deployment pipelines.
 - Release automation.
 - Environment provisioning.
 - Artifact management.
 - GitOps.
 - Deployment strategies.
 - Observability integration.
 - Infrastructure testing.
 - Pipeline governance.
-

5.2 Cloud/Infrastructure Engineer

CloudInfrastructureEngineer

Responsible for:

- GCP infrastructure.
 - Terraform.
 - Networking implementation.
 - IAM implementation.
 - Compute.
 - Storage.
 - Databases.
 - Kubernetes.
 - Infrastructure troubleshooting.
-

5.3 CI/CD Engineer

CICDEngineer

Responsible for:

- Cloud Build / GitHub Actions etc.
- Pipeline templates.
- Build automation.
- Test integration.
- Artifact repositories.
- Promotion workflows.
- Deployment automation.

- Release gates.
-

5.4 SRE / Observability Engineer

SREObservabilityEngineer

Responsible for:

- SLO/SLA/SLI.
 - Monitoring.
 - Logging.
 - Alerting.
 - Tracing.
 - Reliability.
 - Capacity planning.
 - Incident automation.
 - Performance monitoring.
 - Error budgets.
-

5.5 FinOps/Cloud Optimization Engineer

For your GCP Landing Zone project, I would explicitly include this role.

FinOpsEngineer

Responsible for:

- Budgeting.
 - Cost allocation.
 - Labels/tags.
 - Cost dashboards.
 - Cost anomaly detection.
 - BigQuery cost optimization.
 - GKE cost optimization.
 - Storage optimization.
 - Commitment analysis.
 - Idle-resource detection.
 - Cost governance.
 - Unit economics.
-

6. Development Team — 5 Members

Rather than five generic developer agents, create specialized personas.

6.1 Development Lead

DevelopmentLead

Responsibilities:

- Technical implementation leadership.
 - Code standards.
 - Design reviews.
 - Task decomposition.
 - Code review.
 - Developer coordination.
 - Technical debt management.
-

6.2 Backend Engineer

BackendEngineer

Responsibilities:

- APIs.
 - Business logic.
 - Microservices.
 - Database integration.
 - Authentication integration.
 - Event-driven architecture.
 - Performance optimization.
-

6.3 Frontend Engineer

FrontendEngineer

Responsibilities:

- UI.
- UX implementation.
- Frontend architecture.
- API integration.



- Accessibility.
 - Performance.
 - Responsive design.
-

6.4 Integration Engineer

IntegrationEngineer

Responsibilities:

- APIs.
 - Pub/Sub.
 - Event-driven integration.
 - External systems.
 - Data transformation.
 - Messaging.
 - Integration testing.
-

6.5 AI/Automation Engineer

Given the direction of your project, I would make the fifth developer an:

AIAutomationEngineer

Responsibilities:

- AI/LLM integration.
- Agent integration.
- RAG.
- Tool calling.
- AI workflows.
- Prompt engineering.
- Evaluation.
- AI guardrails.
- Agent orchestration.
- AI observability.

This becomes particularly useful if the Landing Zone itself is going to have an **AI-driven operations/governance layer**.

7. Testing Team — 3 Members

Again, make them specialized.

7.1 QA Lead

QALead

Mission: Own overall quality strategy.

Responsibilities:

- Test strategy.
 - Test planning.
 - Quality gates.
 - Test coverage.
 - Defect governance.
 - Release quality assessment.
 - QA metrics.
-

7.2 Automation / Functional Test Engineer

TestAutomationEngineer

Responsibilities:

- Functional testing.
 - API testing.
 - UI testing.
 - Regression testing.
 - Automated test frameworks.
 - Integration testing.
 - End-to-end testing.
-

7.3 Performance / Security / Resilience Test Engineer

NonFunctionalTestEngineer

Responsibilities:

- Load testing.
- Stress testing.
- Performance testing.

- Resilience testing.
- Failover testing.
- Disaster recovery testing.
- Security test coordination.
- Chaos testing.

The Security Architect remains the security authority; this agent executes the required tests.

8. Application Management Group — 3 Members

This group should represent **production ownership**, not just "support".

8.1 Application Management Lead

ApplicationManagementLead

Responsibilities:

- Application operational ownership.
 - Incident management.
 - Problem management.
 - Change management.
 - SLA management.
 - Production readiness.
 - Operational reporting.
 - Escalation management.
-

8.2 L2/L3 Application Support Engineer

ApplicationSupportEngineer

Responsibilities:

- Production troubleshooting.
 - Log analysis.
 - Application diagnostics.
 - API troubleshooting.
 - Database troubleshooting.
 - Incident resolution.
 - Root-cause analysis.
 - Knowledge-base maintenance.
-

8.3 Production/SRE Support Engineer

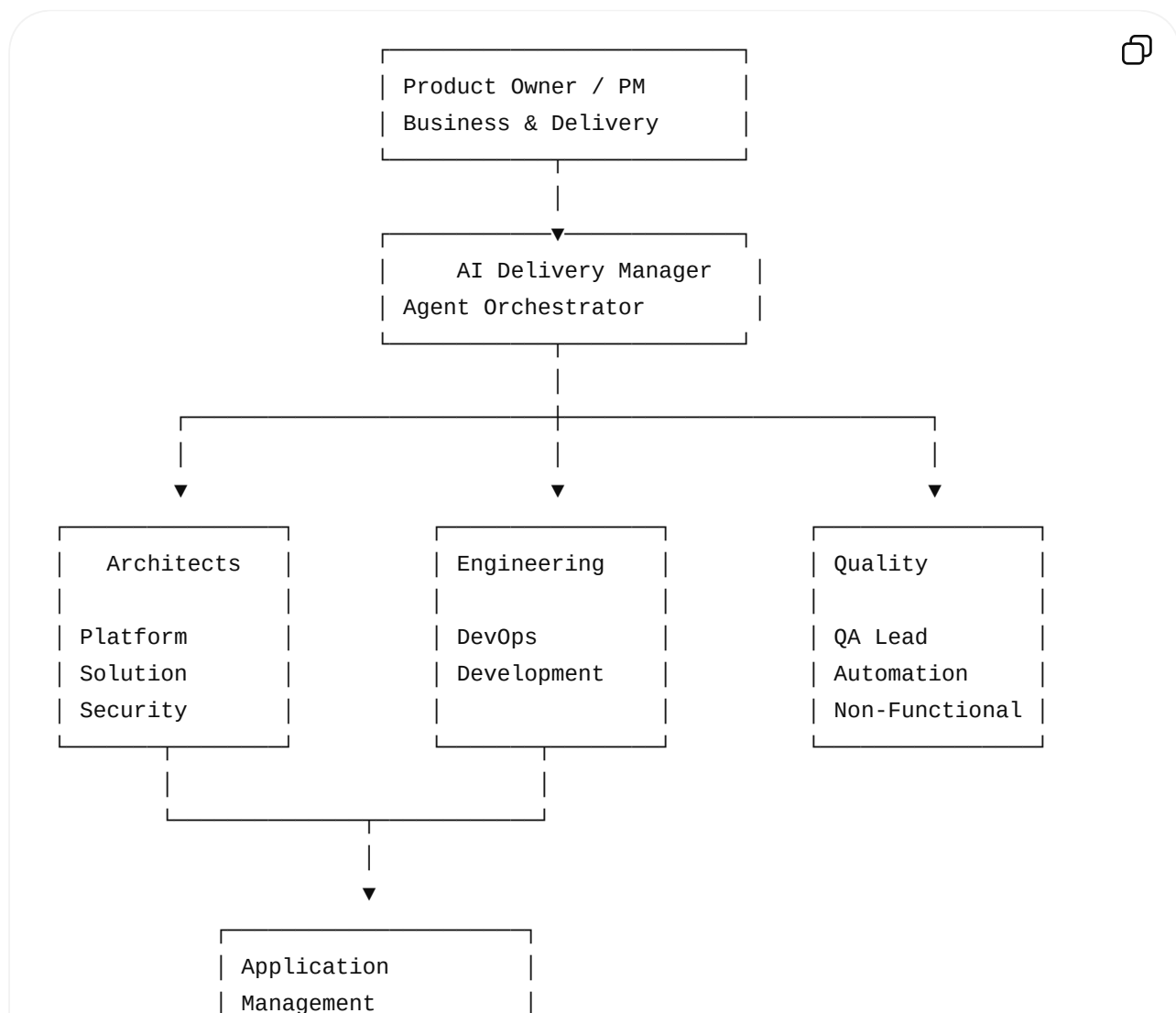
ProductionReliabilityEngineer

Responsibilities:

- Production monitoring.
- Alerts.
- Infrastructure/application correlation.
- Capacity.
- Performance.
- Reliability.
- DR.
- Production deployment validation.
- Operational automation.

Recommended Overall Agent Organization

I would structure the multi-agent system like this:



AM Lead
L2/L3 Support
Production/SRE

But I would add one important layer

For a **fully autonomous multi-agent engineering organization**, don't let all agents communicate directly with everyone.

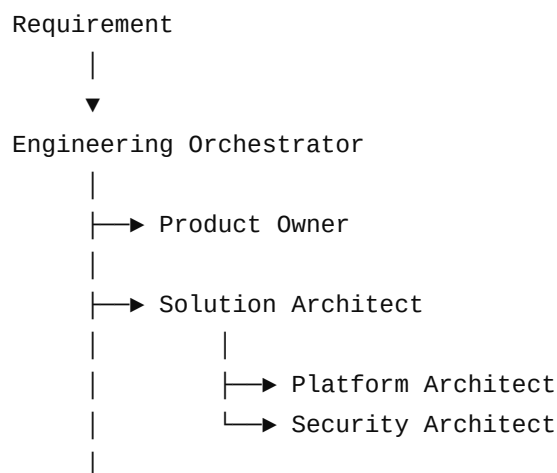
Introduce an **Engineering Orchestrator / Chief Architect Agent**.

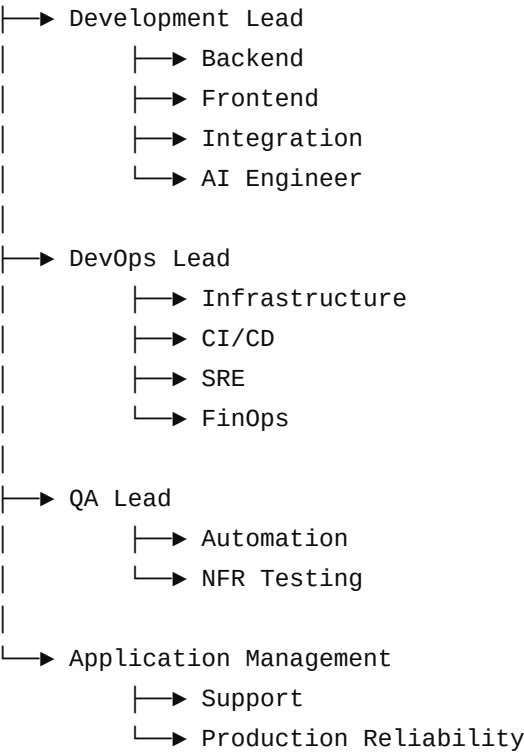
EngineeringOrchestrator

Its job is to:

1. Receive the requirement.
2. Determine which agents need to participate.
3. Create the work breakdown.
4. Assign tasks.
5. Maintain shared project context.
6. Track dependencies.
7. Collect decisions.
8. Detect conflicts.
9. Escalate architectural disagreements.
10. Coordinate reviews.
11. Enforce quality gates.
12. Ensure security approval.
13. Ensure testing approval.
14. Ensure production readiness.
15. Produce the final delivery package.

For example:





Agent-to-Agent Governance

The most important part is **not the number of agents**. It is their **authority model**.

I recommend:

Area	Final Authority
Business priority	Product Owner
Scope	Product Owner
Delivery	PM
Overall solution	Solution Architect
GCP platform	Platform Architect
Security	Security Architect
Code implementation	Development Lead
Infrastructure implementation	DevOps Lead
Testing quality	QA Lead
Production operations	Application Management Lead

Area	Final Authority
Cloud cost	FinOps Engineer
Technical conflict	Engineering Orchestrator + relevant Architect
Security conflict	Security Architect
Business/technical conflict	Product Owner + Solution Architect

Critical rule

Agents should be allowed to disagree.

For example:

Solution Architect:
"Use GKE."

Platform Architect:
"Approved."

Security Architect:
"Container security controls required."

DevOps:
"Implementation feasible."

FinOps:
"GKE is 35% more expensive than Cloud Run for this workload."

QA:
"Performance requirement requires GKE."

Solution Architect:
"Decision: GKE."

Engineering Orchestrator:
"Record ADR-017."

That is much closer to a **real engineering organization** than a collection of independent AI chatbots.

Recommended Agent Count

Your original structure gives you approximately:

8 functional groups → 19+ agents

I would evolve it to approximately **25–27 specialized agents**:

- 1 Product Owner
- 1 Project Manager / Delivery Manager
- 1 Engineering Orchestrator
- 1 Platform Architect
- 1 Solution Architect
- 1 Security Architect
- 1 DevOps Lead
- 1 Infrastructure Engineer
- 1 CI/CD Engineer
- 1 SRE/Observability Engineer
- 1 FinOps Engineer
- 1 Development Lead
- 1 Frontend Engineer
- 1 Backend Engineer
- 1 Integration Engineer
- 1 AI/Automation Engineer
- 1 QA Lead
- 1 Test Automation Engineer
- 1 NFR/Performance Engineer
- 1 Application Management Lead
- 1 Application Support Engineer
- 1 Production Reliability Engineer

And optionally:

- Data Architect
- Database Engineer
- Compliance/Governance Specialist
- Technical Writer/Documentation Agent

For the GCP Landing Zone specifically, I would make the next step defining the detailed persona specification for each agent — including system prompt, responsibilities, tools, memory, inputs/outputs, authority, escalation rules, collaboration protocol, and Definition of Done. That can then become the actual blueprint for implementing the multi-agent team in LangGraph, ADK, Semantic Kernel, or your preferred framework.



That error means you're running the script with `sh bootstrap.sh` (or your environment maps `sh` to a shell such as `dash`). `pipefail` is a **Bash** option, not POSIX `sh`.

Quick fix

Run it explicitly with Bash:

```
</> Bash
bash bootstrap.sh
```



or:

```
</> Bash
chmod +x bootstrap.sh
./bootstrap.sh
```



Don't run:

```
</> Bash
sh bootstrap.sh
```



Verify Bash

```
</> Bash
bash --version
```



If Bash is installed, the script should start normally.

I recommend making the bootstrap more robust

Change the first lines from:

```
</> Bash
#!/usr/bin/env bash
set -Eeuo pipefail
```



to:

```
</> Bash
#!/usr/bin/env bash

# Refuse to run under a non-Bash shell
if [ -z "${BASH_VERSION:-}" ]; then
```



```
    echo "ERROR: This script must be run with Bash."
    echo "Use: bash bootstrap.sh"
    exit 1
fi

set -Eeuo pipefail
```

Then even if someone accidentally does:

```
</> Bash

sh bootstrap.sh
```



they'll get a clear message instead of:

```
set: Illegal option -o pipefail
```



Also check the file format

Because you're likely running this on Linux, make sure it doesn't have Windows CRLF line endings:

```
</> Bash

file bootstrap.sh
```



If it says CRLF , fix it with:

```
</> Bash

sed -i 's/\r$//' bootstrap.sh
```



Then:

```
</> Bash

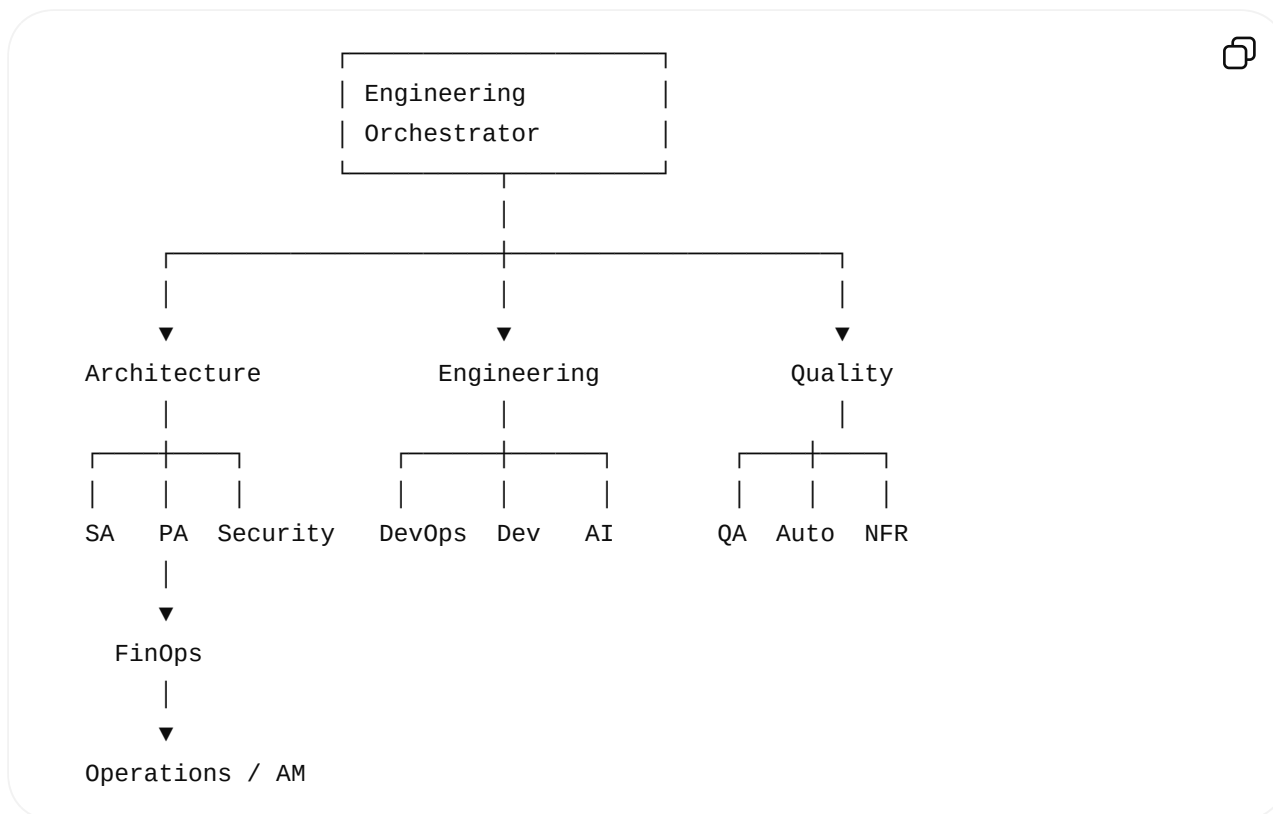
chmod +x bootstrap.sh
./bootstrap.sh
```



One more thing

The bootstrap I gave you is intentionally **Phase 1**. Once it runs, the next stage should be to make the script automatically build the **full 22-agent LangGraph organization**, rather than just creating the scaffolding.

That next version should generate:



with **agent-to-agent delegation, parallel execution, shared state, RAG, memory, tool authorization, human approval gates, Git integration, Terraform integration, and eventually GCP/NVIDIA NIM deployment.**



Create the script for next stage

Yes. The next stage should **build the actual 22-agent LangGraph organization**, not just create folders.

I'll make the script **idempotent** so you can run it on top of the bootstrap project without destroying existing work. It will add the agent registry/factory, specialist personas, orchestrator routing, architecture/development/testing/operations subgraphs, shared state, tool registry, approval gates, and a CLI.

I'll generate it as a downloadable `stage2_build_team.sh`.



which model are we using now `NVIDIA_API_KEY=nvapi-REPLACE_ME`

Right now, **the** `NVIDIA_API_KEY=nvapi-REPLACE_ME` **is only a placeholder**. It does not select a model by itself.

In the bootstrap we created, the **actual model is selected by** `config/models.yaml` , and the current default Orchestrator is configured to use:

Current Orchestrator model

```
nvidia/nemotron-3-ultra-550b-a55b
```

That's the model used here:

</> YAML

```
architecture_critical:
  provider: nvidia
  model: nvidia/nemotron-3-ultra-550b-a55b
```

and the Orchestrator requests:

</> Python

```
get_model("architecture_critical")
```

Run

NVIDIA currently lists **Nemotron 3 Ultra 550B A55B** as a free API endpoint and describes it as optimized for agentic reasoning, coding, planning, tool calling, and long-context workloads.



[NVIDIA Nemotron 3 Ultra on Build ↗](#)

Our planned model allocation

Agent workload	Model
Engineering Orchestrator	Nemotron 3 Ultra 550B A55B
Solution Architect	Nemotron 3 Ultra
Platform Architect	Nemotron 3 Super 120B A12B
Security Architect	Nemotron 3 Super
Product Owner / PM	Nemotron 3 Super
DevOps Lead	Nemotron 3 Super

Agent workload	Model
SRE	Nemotron 3 Super
QA Lead	Nemotron 3 Super
Application Management	Nemotron 3 Super
Backend / Frontend development	DeepSeek V4 Pro
Terraform / Infrastructure	DeepSeek V4 Pro
Integration	DeepSeek V4 Pro
AI/Automation	DeepSeek V4 Pro
Routine/fast agents	Nemotron Lightning
Fast coding/testing	DeepSeek V4 Flash
Embeddings/RAG	Nemotron 3 Embed 1B
Reranking	Llama Nemotron Rerank 1B V2

NVIDIA currently lists **Nemotron 3 Super 120B A12B** as a free endpoint with 1M context and capabilities specifically covering reasoning, coding, planning and tool calling.  [NVIDIA NIM APIs](#)

NVIDIA also currently lists **DeepSeek V4 Pro** as a free endpoint optimized for coding, reasoning and agentic workloads.  [NVIDIA NIM APIs](#)

One important correction

Before we proceed with Stage 2, I would **not blindly keep the original model list**. NVIDIA's catalog has changed since we designed the first bootstrap. For example, the current catalog now shows **Nemotron 3 Nano 30B A3B** as another available model, and the current catalog confirms the exact Nemotron/DeepSeek identifiers above.  [NVIDIA NIM APIs +1](#)

So I recommend that **Stage 2 dynamically validates the configured NVIDIA models at startup** rather than failing later because a model identifier changed.

Your `.env` should simply contain your real key:

```
</> env

NVIDIA_API_KEY=nvapi-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The key authenticates you; `modelSelector.yaml` **determines which model is called.**

If you're ready, I'll make the Stage-2 bootstrap script with this model architecture and add **automatic model validation + model fallback**, so the 22-agent team doesn't break when NVIDIA changes endpoint availability.



ChatGPT can make mistakes. Check important info.